12주차

서브쿼리
두 단계 질문을 한 문장으로

- 스칼라 서브쿼리 — "평균보다 오래 입원한"
- IN 서브쿼리 — 명단으로 거르기
- EXISTS / NOT EXISTS — "~한 적이 있는가/없는가"
- NOT IN의 NULL 함정

> **이번 주 대표 질문**
> 항생제를 한 번도 처방받지 않은 환자는?

서브쿼리 — 두 단계 질문을 한 문장으로

개념

임상 질문의 실제 구조

"평균보다 오래 입원한 환자는?"

= ① **먼저** 평균 재원일수를 구하고

② 그 값보다 큰 행을 찾아라

SQL의 대응

괄호 안의 쿼리(①)가 **먼저 실행**되어 값·명단이 되고, 바깥 쿼리(②)가 그 결과를 조건으로 사용합니다.

서브쿼리가 놓이는 자리: WHERE(오늘의 중심), FROM(지난주 이미 사용), SELECT

스칼라 서브쿼리 — 값 하나를 조건에 꽂기

대표 질문

임상 질문

평균 재원일수보다 오래 입원한 환자는?

```
SELECT visit_id, patient_id,
       julianday(discharge_date)
         - julianday(visit_date) AS los
FROM visit
WHERE visit_type = '입원'
  AND julianday(discharge_date)
        - julianday(visit_date) >
      (SELECT AVG(julianday(discharge_date)
                - julianday(visit_date))
       FROM visit
       WHERE visit_type = '입원')
ORDER BY los DESC;
```

visit_id	patient_id	los
V0089	P039	35
V0073	P027	20
V0061	P021	19
V0069	P025	19
V0077	P029	17
V0075	P028	14

… 총 7행 중 6행 표시

괄호 안이 먼저 평균(11.7일)을 계산 → 바깥 쿼리가 그보다 긴 입원만 필터. 35일 입원한 P039(대퇴골 골절)가 선두입니다. **기준값을 데이터가 스스로 정하므로** 평균이 바뀌어도 쿼리를 수정할 필요가 없습니다.

IN 서브쿼리 — 명단을 만들어 거르기

임상 질문

뇌경색(I63) 진단을 받은 적 있는 환자의 인적 정보

```sql
SELECT patient_id, sex, birth_date
FROM patient
WHERE patient_id IN (
    SELECT v.patient_id
    FROM visit v
    JOIN diagnosis d
      ON v.visit_id = d.visit_id
    WHERE d.kcd_code LIKE 'I63%');
```

patient_id	sex	birth_date
P021	M	1969-01-02
P022	F	1970-08-24
P023	F	1969-06-14
P024	M	1969-04-28
P025	M	1965-01-15
P026	M	1964-03-16

… 총 10행 중 6행 표시

안쪽 쿼리가 'I63 진단 이력자 명단'을 만들고, 바깥 쿼리가 그 명단에 든 환자만 통과. **"~한 적이 있는 환자군"**은 코호트 정의의 기본 문형입니다(다음 주 주인공).

NOT EXISTS — "~한 적이 전혀 없다"

대표 질문

임상 질문

항생제를 한 번도 처방받지 않은 환자는 몇 명인가?

```sql
SELECT COUNT(*) AS 무항생제환자수
FROM patient p
WHERE NOT EXISTS (
    SELECT 1
    FROM visit v
    JOIN prescription rx
      ON rx.visit_id = v.visit_id
    WHERE v.patient_id = p.patient_id
      AND rx.drug_code IN
          ('CTX2','AMC625','LVX500'));
```

무항생제환자수
41

총 1행

EXISTS는 '한 건이라도 있는가', NOT EXISTS는 '한 건도 없는가'를 묻습니다. 안쪽의 `v.patient_id = p.patient_id` 처럼 바깥 행과 연결된 형태를 **상관 서브쿼리**라 부릅니다 — 환자 한 명 한 명에 대해 질문을 반복하는 그림.

EXISTS의 짝 — 노출군 정의

임상 질문

반대로: 항생제 노출 이력이 있는 환자와 그 약물

```
SELECT DISTINCT v.patient_id,
       rx.drug_name
FROM visit v
JOIN prescription rx
  ON rx.visit_id = v.visit_id
WHERE rx.drug_code IN
      ('CTX2','AMC625','LVX500')
ORDER BY v.patient_id;
```

patient_id	drug_name
P031	세프트리악손 주사
P032	세프트리악손 주사
P033	세프트리악손 주사
P034	세프트리악손 주사
P035	세프트리악손 주사
P036	세프트리악손 주사

… 총 9행 중 6행 표시

약물 노출군 vs 비노출군 — 약물역학 연구의 두 팔이 EXISTS / NOT EXISTS 한 쌍으로 정의됩니다.

NOT IN의 함정 — NULL이 끼면 전부 사라진다

함정

```
-- 위험한 형태
SELECT * FROM patient
WHERE patient_id NOT IN
  (SELECT patient_id FROM some_list);
-- some_list의 patient_id에 NULL이
-- 하나라도 있으면 → 결과 0행!
```

왜?

- `x NOT IN (a, b, NULL)`은 `x≠a AND x≠b AND x≠NULL`
- NULL과의 비교는 참도 거짓도 아닌 **'모름'** → 전체가 '모름' → 아무도 통과 못 함
- 에러도 없이 **조용히 0행** — 최악의 실패 유형

처방

- 결측 가능성이 있으면 **NOT EXISTS를 기본기로**

서브쿼리 vs JOIN — 언제 무엇을

서브쿼리가 자연스러운 질문

- "평균/최댓값 **기준**으로" — 스칼라
- "~인 적 있는/없는 **사람**" — EXISTS류
- 결과에 상대 테이블 컬럼이 **필요 없을 때**

JOIN이 자연스러운 질문

- 양쪽 테이블의 컬럼을 **나란히 출력**해야 할 때
- "진단명과 검사값을 한 줄에"
- 여러 테이블의 속성을 조합한 계산

둘은 상당 부분 서로 변환 가능하며, DBMS가 내부적으로 같은 실행계획으로 바꾸기도 합니다 — SQLite에서는 `EXPLAIN QUERY PLAN` 을 쿼리 앞에 붙이면 실행 방식을 확인할 수 있습니다.

연습문제

실습

- **Q1.** 평균 투약일수보다 길게 처방된 약물 행은?

  스칼라 서브쿼리

- **Q2.** 폐렴(J18) 진단 이력이 있는 환자의 성별·생년월일은?

  IN

- **Q3.** 검사(lab)를 한 번도 받지 않은 환자 수는?

  NOT EXISTS

- **Q4.** 자기 진료과 평균보다 재원이 길었던 입원 건은?

  도전: 상관 서브쿼리

- **Q5.** Q3를 NOT IN으로도 써 보고, 어떤 상황에서 위험해지는지 한 줄로 설명하라

이번 주 용어 정리

용어

용어	뜻
서브쿼리(subquery)	괄호 안에서 먼저 실행되는 내부 쿼리
스칼라 서브쿼리	값 하나를 반환하는 서브쿼리 — 비교 기준으로 사용
상관 서브쿼리	바깥 쿼리의 행을 참조하며 행마다 다시 평가되는 서브쿼리
EXISTS / NOT EXISTS	조건에 맞는 행이 하나라도 있는가 / 전혀 없는가
IN / NOT IN	명단 포함 / 미포함 — NOT IN은 명단에 NULL이 있으면 0행
노출군 / 비노출군	약물 등 연구 요인에 노출된 집단 / 노출되지 않은 대조 집단

12주차 정리

- 서브쿼리 = **두 단계 질문의 한 문장화** — 안쪽이 먼저, 바깥이 나중
- 스칼라(값 하나) · IN(명단) · **EXISTS/NOT EXISTS(이력 유무)**
- "~한 적 없는 환자"는 NOT EXISTS — **NOT IN은 NULL이 섞이면 조용히 0행**이 된다
- 노출군/비노출군 정의 = EXISTS 한 쌍 — 약물역학의 문법

과제 Colab 12주차 Q1~Q5

다음 주 종합 실전 — 연구 질문에서 코호트 추출, 그리고 Table 1까지 한 번에